What is Natural Language Processing?

1. Natural Language Processing is a subfield of artificial intelligence concerned with methods of communication between computers and natural languages such as English, Hindi, etc.

2. It is an intersection of fields of Computer Science, linguistics, and AI

3. Objective of Natural Language processing is to perform useful tasks involving human languages like:

    - Sentiment Analysis
    - Machine Translation
    - Part of Speech Tags
    - Human-Machine communication (chat-bots)

Importance of Natural Language Processing:

1. It is the most widely used AI technology in the world.
2. It is used in various applications like:
    - Search Engines
    - Chatbots
    - Virtual Assistants
    - Language Translation
    - Sentiment Analysis
3. It is used in various industries like:
    - Healthcare
    - Finance
    - E-commerce
    - Social Media
4. It is used to analyze and understand human language, which is a complex and unstructured data, and to extract useful information from it.

Different tasks in Natural Language Processing?

1. **Text Classification**: It is the task of assigning a label to a given text. For example, classifying an email as spam or not spam.

2. **Named Entity Recognition**: It is the task of identifying and classifying named entities in a text. For example, identifying and classifying names of people, organizations, and locations in a news article.

3. **Part of Speech Tagging**: It is the task of assigning a part of speech tag to each word in a text. For example, identifying whether a word is a noun, verb, adjective, etc.

    Detailed Definition:

    In corpus linguistics, part-of-speech tagging (POS tagging or PoS tagging or POST), also called grammatical tagging or word-category disambiguation, is the process of marking up a word in a text (corpus) as corresponding to a particular part of speech, based on both its definition and its context.

    A simplified form of this is the identification of words as nouns, verbs, adjectives, adverbs, etc.

Example: "She sells seashells on the seashore"

- She → PRP (Personal Pronoun)
- sells → VBZ (Verb, 3rd person singular present)
- seashells → NNS (Noun, plural)
- on → IN (Preposition)
- the → DT (Determiner)
- seashore → NN (Noun, singular)

Tagset Reference: Penn Treebank POS Tags

4. **Sentiment Analysis**: It is the task of determining the sentiment expressed in a text. For example, determining whether a product review is positive, negative, or neutral.

5. **Machine Translation**: It is the task of translating a text from one language to another. For example, translating a news article from English to Hindi.

6. **Text Summarization**: It is the task of generating a summary of a given text. For example, generating a summary of a news article.

7. **Question Answering**: It is the task of answering a question based on a given text. For example, answering a question about a news article based on the information present in the article.

Text Pre-Processing

Text Pre-Processing is the process of cleaning and preparing the text data for analysis. It is an important step in Natural Language Processing as it helps to improve the performance of the models by removing noise and irrelevant information from the text data.

Steps involved in Text Pre-Processing:

1. **Tokenization**: It is the process of breaking down a text into individual words or tokens. For example, breaking down the sentence "I love Natural Language Processing" into individual words "I", "love", "Natural", "Language", "Processing". **Note:** we could also use the Regexp Tokenizer to break down the text into tokens based on a regular expression pattern. For example, we could use the pattern "\w+" to break down the sentence into individual words while ignoring punctuation marks.

2. **Stop Words Removal**: It is the process of removing common words that do not add much meaning to the text. For example, removing words like "the", "is", "in", etc. from the text data.

3. **Stemming**: It is the process of reducing a word to its root form. For example, reducing the word "running" to "run".

4. **Lemmatization**: It is the process of reducing a word to its base form. For example, reducing the word "running" to "run" and "better" to "good". - More important than stemming as it considers the context and meaning of the word while reducing it to its base form resolving it to the dectionary form of the word.

   Key Differences between Stemming and Lemmatization:

Stemming:

- Crude/heuristic approach - chops off word endings using simple rules
- May produce non-words (e.g., "caring" → "car", "studies" → "studi")
- Faster - simple rule-based algorithm
- Context-independent - doesn't consider part of speech
- Example: "better" → "better" (unchanged)

Lemmatization:

- Linguistic approach - uses vocabulary and morphological analysis
- Always produces valid dictionary words
- Slower - requires dictionary lookup and linguistic rules
- Context-aware - considers part of speech and meaning
- Example: "better" → "good", "am/are/is" → "be"

When to use which:

- Use Stemming: When speed is important, approximate matching acceptable, search engines
- Use Lemmatization: When accuracy is critical, need valid words, text analytics/NLP models

5. **Removing Stop Words**: It is the process of removing common words that do not add much meaning to the text. For example, removing words like "the", "is", "in", etc. from the text data.

6. **Removing Punctuation**: It is the process of removing punctuation marks from the text data. For example, removing punctuation marks like ".", ",", "!", etc. from the text data.

7. **Removing Numbers**: It is the process of removing numbers from the text data. For example, removing numbers like "1", "2", "3", etc. from the text data.

8. **Lowercasing**: It is the process of converting all the characters in the text data to lowercase. For example, converting the sentence "I Love Natural Language Processing" to "i love natural language processing".

9. **Removing Extra Spaces**: It is the process of removing extra spaces from the text data. For example, removing extra spaces from the sentence "I love Natural Language Processing" to "I love Natural Language Processing".

10. **Removing URLs**: It is the process of removing URLs from the text data. For example, removing URLs like "https://www.example.com" from the text data.

11. **Removing HTML Tags**: It is the process of removing HTML tags from the text data. For example, removing HTML tags like `"<p>", "<a>"`, etc. from the text data.

12. **Removing Special Characters**: It is the process of removing special characters from the text data. For example, removing special characters like "@", "#", "$", etc. from the text data.

13. **Removing Emojis**: It is the process of removing emojis from the text data. For example, removing emojis like "😊 ", "😂 ", etc. from the text data.

14. **Removing Non-ASCII Characters**: It is the process of removing non-ASCII characters from the text data. For example, removing non-ASCII characters like "é", "ü", etc. from the text data.

15. **Removing Accents**: It is the process of removing accents from the text data. For example, removing accents from the word "café" to "cafe".
16. **Removing Duplicates**: It is the process of removing duplicate entries from the text data. For example, removing duplicate entries from a list of product reviews.
17. **Removing Non-Textual Data**: It is the process of removing non-textual data from the text data. For example, removing images, videos, etc. from the text data.

Stop Word Removal

Stop words are common words that do not add much meaning to the text. They are often removed from the text data to improve the performance of the models.

Examples of stop words: "the", "is", "in", "a", "an", "and", "or", "but", "I", "you", "he", "she", "at", "by", "for", "with", "from", "to", etc.

For example, in the sentence "I love Natural Language Processing", only "I" is a stop word that can be removed. The words "love", "Natural", "Language", "Processing" are content words that carry important meaning and should be kept.

When using bag of words or TF-IDF vectorization, stop words are often removed to reduce the dimensionality of the feature space and to focus on the more meaningful words in the text data.

It is to note that - it is not always a good idea to remove stop words. In some cases, stop words can carry important meaning and should be kept. For example, in the sentence "I am not happy", the word "not" is a stop word but it carries important meaning and should be kept. Therefore, it is important to carefully consider whether to remove stop words or not based on the specific use case and the nature of the text data.

📚 IMPORTANT: Key Preprocessing Concepts to Study

> **In Natural Language Processing (NLP), preprocessing is a crucial first step that transforms raw text data into a format suitable for analysis by machines. It's like cleaning and organizing your data before you can actually use it. Here's a breakdown of the key preprocessing techniques:**
>
> **1. Text Cleaning:** This involves getting rid of unwanted elements in the text that don't contribute to meaning. Examples include: Removing punctuation (Commas, periods, question marks, etc.), Removing special characters (Symbols, emojis), and Handling HTML tags or extra spaces.
>
> **2. Tokenization:** This breaks down the text into smaller, more manageable units. Types include: Word tokenization (splits text into individual words), Sentence tokenization (splits text into sentences), and N-gram tokenization (creates sequences of n words like bigrams, trigrams).
>
> **3. Stopword Removal:** Stopwords are very common words like "the," "a," "an," "is," etc., that carry little meaning on their own. Removing them can improve efficiency and focus analysis on more content-rich words.

> **4. Stemming and Lemmatization:** These techniques aim to reduce words to their base form. Stemming uses a set of rules to chop off suffixes (playing → play, running → run) but can be imprecise. Lemmatization uses a dictionary to map words to their root form (considering context) and is generally more accurate but computationally expensive.
>
> **5. Normalization:** This involves transforming text into a consistent format. Examples include: Lowercasing (converting all text to lowercase), and Handling numbers (keep, convert to text, or remove depending on analysis).
>
> 💡 **Key Takeaway:** By applying these preprocessing techniques, you prepare your text data for downstream NLP tasks like sentiment analysis, topic modeling, or machine translation. The specific techniques you use will depend on your specific NLP application and the nature of your data.

💻 Coding Reference

📓 **Hands-on Practice Notebooks**:

1. Into_NLP_Unit1.0-text-preprocessing.ipynb - Complete text preprocessing pipeline with tokenization, stop words removal, stemming, and lemmatization
2. text normalization (1).ipynb - Text normalization techniques and examples

These notebooks contain complete code examples for all text preprocessing techniques with detailed explanations and practice exercises.

Count Vectorization and TF-IDF Vectorization

what is Count Vectorization?

Count Vectorization is a technique used to convert text data into a numerical format that can be used by machine learning algorithms. It works by counting the number of times each word appears in a document and creating a vector representation of the document based on these counts. The resulting vector is called a "bag of words" representation, where the order of the words is not considered, only their frequency.

Bag of Words Model

Simple Explanation:

The Bag of Words (BoW) model converts text into numbers by:

1. **Ignoring word order and grammar** - only counts how many times each word appears
2. **Creating a vocabulary** - a list of all unique words from all documents
3. **Counting frequencies** - each document becomes a vector showing word counts
4. **Used for classification** - helps machines understand and classify text

Example:

Raw Text: "it is a puppy and it is extremely cute"

Bag-of-words vector representation:

```
it        → 2
they      → 0
puppy     → 1
and       → 1
cat       → 0
aardvark  → 0
cute      → 1
extremely → 1
...       → ...
```

Note: The vocabulary size can be very large (thousands of words), and most entries will be 0 for any given document.

```python
from sklearn.feature_extraction.text import CountVectorizer

# list of text documents
text = ["The quick brown fox jumped over the lazy dog."]

# create the transform
vectorizer = CountVectorizer()

# tokenize and build vocab
vectorizer.fit(text)

# summarize
print(vectorizer.vocabulary_)

# encode document
vector = vectorizer.transform(text)

# summarize encoded vector
print(vector.shape)
print(type(vector))
print(vector.toarray())
```

Output:

```
{'the': 7, 'quick': 6, 'brown': 0, 'fox': 2, 'jumped': 3, 'over': 5,
'lazy': 4, 'dog': 1}
(1, 8)
<class 'scipy.sparse.csr.csr_matrix'>
[[1 1 1 1 1 1 1 2]]
```

Explanation:

- The vocabulary dictionary maps each unique word to an index
- The vector shape (1, 8) means 1 document with 8 unique words
- The output is a sparse matrix for memory efficiency
- The array `[1 1 1 1 1 1 1 2]` shows word counts: 'the' appears 2 times (index 7), all others appear once

Example 2: Encoding another document with the same vocabulary:

```
# encode another document
text2 = ["the puppy"]
vector = vectorizer.transform(text2)
print(vector.toarray())
```

Output:

```
[[0 0 0 0 0 0 0 1]]
```

Explanation:

- The same vocabulary from the first document is used
- Only "the" (index 7) appears in the new text, so it gets count 1
- "puppy" is not in the original vocabulary, so it's ignored
- All other positions get 0 (words not present in this document)
- This shows how CountVectorizer maintains a fixed vocabulary across documents

what is TF-IDF Vectorization?

TF-IDF (Term Frequency-Inverse Document Frequency) Vectorization is a technique used to convert text data into a numerical format that can be used by machine learning algorithms. It works by calculating the term frequency (TF) of each word in a document, which is the number of times the word appears in the document, and the inverse document frequency (IDF) of each word, which is a measure of how important the word is in the entire corpus. The resulting vector is called a "TF-IDF" representation, where each word is weighted based on its importance in the document and the corpus.

TF-IDF Vectors - Detailed Explanation

Key Points:

- TF-IDF (term frequency times inverse document frequency) is a scheme to weight individual tokens
- One advantage of TF-IDF is to **reduce the impact of tokens that occur very frequently**, hence offering little to none in terms of information

Mathematical Formula:

$$TFIDF \text{ score for term } i \text{ in document } j = TF(i,j) \times IDF(i)$$

Where:

$$TF = \text{Term Frequency}$$

$$IDF = \text{Inverse Document Frequency}$$

Term Frequency (TF):

$$TF(i,j) = \frac{\text{Term i frequency in document j}}{\text{Total words in document j}}$$

Inverse Document Frequency (IDF):

$$IDF(i) = \log_2\left(\frac{\text{Total documents}}{\text{documents with term i}}\right)$$

Variables:

- t = Term
- j = Document

Example of TF-IDF Vectorization:

Manual Calculation Example

Given Documents:

- **Document #1**: "He is a good boy. She is also good."
- **Document #2**: "Radhika is a good person."

Step 1: Calculate Term Frequency (TF)

Word Counts:

Word	Doc #1 Count	Word	Doc #2 Count
He	1	Radhika	1
is	2	is	1
a	1	a	1
good	2	good	1
boy	1	person	1
she	1		
also	1		
Total	**9**	**Total**	**5**

TF Formula:

$$TF = \frac{\text{Frequency of the word in a Doc}}{\text{Total number of words in the Doc}}$$

TF Calculations:

Word	TF(word, doc#1)	TF(word, doc#2)
He	1/9 = 0.11	0/5 = 0
good	2/9 = 0.22	1/5 = 0.2

> **Note:** TF captures how important a word is to the document (without looking at other documents in the dataset)

Step 2: Calculate Inverse Document Frequency (IDF)

IDF Formula:

$$IDF = \log\left(\frac{\text{Num of Docs}}{\text{Word in Num of Docs}}\right)$$

IDF Calculations:

Word	Calculation	IDF Value
He	log(2/1)	0.301
good	log(2/2)	0

> **Note:** IDF tells us if a word (feature) can be used to distinguish documents. If a word appears in majority of the documents then IDF will be close to '0' i.e. give low weightage to that feature.

Step 3: Calculate TF-IDF Scores

TF-IDF = TF × IDF

TF-IDF Calculations (TF × IDF):

Word	TF-IDF(word, doc#1)	TF-IDF(word, doc#2)
He	0.11 × 0.301 = 0.03311	0 × 0.301 = 0
good	0.22 × 0 = 0	0.2 × 0 = 0

Step 4: TF-IDF Vector Representation

Vocabulary (from both documents): a, also, boy, good, He, is, person, She, Radhika

Final TF-IDF Vector Table:

	a	also	boy	good	He	is	person	She	Radhika
Index	0	1	2	3	4	5	6	7	8
Doc #1				0	0.03311				
Doc #2				0	0				

Key Insight:

- Word "good" appears in both documents → IDF = 0 → TF-IDF = 0 (not useful for distinguishing)
- Word "He" appears only in doc#1 → Higher IDF → Non-zero TF-IDF (useful for distinguishing)

Python Implementation Example

```python
from sklearn.feature_extraction.text import TfidfVectorizer

# list of text documents
text = ["The quick brown fox jumped over the lazy dog.",
        "The dog.",
        "The fox"]

# create the transform
vectorizer = TfidfVectorizer()

# tokenize and build vocab
vectorizer.fit(text)

# summarize
print(vectorizer.vocabulary_)
print(vectorizer.idf_)

# encode document
vector = vectorizer.transform([text[0]])

# summarize encoded vector
print(vector.shape)
print(vector.toarray())
```

Output:

```
{'the': 7, 'quick': 6, 'brown': 0, 'fox': 2, 'jumped': 3, 'over': 5,
'lazy': 4, 'dog': 1}
[1.69314718 1.28768207 1.28768207 1.69314718 1.69314718 1.69314718
 1.69314718 1.        ]
(1, 8)
[[0.36388646 0.27674503 0.27674503 0.36388646 0.36388646 0.36388646
  0.36388646 0.42983441]]
```

Explanation:

- **Vocabulary**: Same as CountVectorizer - maps each unique word to an index based on the alphabetical order
- **IDF values**:
  - "the" has IDF = 1.0 (lowest) because it appears in all 3 documents (most common)

    - "quick", "brown", "jumped", "over", "lazy" have highest IDF (~1.69) because they appear in only 1 document (most distinctive)
    - "dog" and "fox" have IDF ~1.29 (medium) because they appear in 2 out of 3 documents
- **TF-IDF scores**: Lower values for common words ("the" = 0.43), higher values for distinctive words (0.36-0.37)
- **Key insight**: TF-IDF automatically downweights common words and highlights important/distinctive words, unlike CountVectorizer which treats all words equally